Desafio técnico

Intelligence & Analitycs

DATA DRIVEN INDEX

Agenda: Produto & Métricas · Modelagem · SQL · Visualização · Avaliação

Agenda

01	Introdução
02	Produto – Métricas – Governança
03	Modelagem – SQL – Visualização
04	Avaliação
05	Encerramento

GitHub

O repositório está disponível em:

https://github.com/ersjunior/case_meli_ddi

Desafio Analista de Dados – Mercado Livre

Contexto do Desafio

Reformular o relatório do **Data Driven Index (DDI)**:
Índice que mede o quanto as equipes são orientadas a dados.
O trabalho envolve diagnóstico do dashboard atual, definição de métricas, modelagem de dados, escrita de queries SQL e criação de um mockup de dashboard.

Os dados são fictícios:
(800 colaboradores, 10 sites, 12 divisões, jan–dez 2025)
Fornecidos em dois CSVs.

Solução Proposta – DDI Reformulado

A solução abrange cinco frentes:

- Diagnóstico das lacunas do dashboard atual
- Criação de um dicionário de métricas com regras claras de elegibilidade
- Modelagem em camadas Silver/Gold para histórico organizacional
- Queries SQL para calcular:
  colaboradores ativos, usuários potenciais e *data-driven* users
- Mockup de dashboard com navegação hierárquica
  empresa → site → divisão, mostrando variações MoM e tendências.

Produto

Universo base	`Colaboradores ativos no mês M`	Alinhado ao enunciado: "apenas colaboradores ativos".
Ativo no mês	`start_date ≤ último dia de M` `E (end_date IS NULL OU end_date ≥ 1º dia de M)`	end_date nulo = ainda na empresa. Saída mid-month = ativo no mês civil inteiro (ajustável para >15 dias se RH definir).
Exclusão	`division = 'Shipping'` `AND function = 'Representative'`	Texto literal do case. Ambos os critérios necessários para não excluir outros papéis de Shipping.
ORG no mês	`Último tim_day ≤ último dia de M` `(SCD tipo 2 por mês)`	Congela site/divisão/dept para o mês. Trocas mid-month são resolvidas deterministicamente.
Sem DDI no mês	`Classificação = 'Sem registro DDI'` `entra no denominador de coverage_rate`	Ativo elegível sem linha em tb_ddi não pode ser ignorado — ele distorceria a taxa de adoção.
Dedup DDI	`DISTINCT ON (username, date_month)` `ORDER BY is_data_user DESC`	Regra determinística: preserva a linha TRUE em caso de conflito; documentada em stg_ddi.

Spec Completo (7 métricas)

active_collaborators
Grão: dept-mês

```
COUNT(DISTINCT username) após filtros de elegibilidade
```

potential_users
Grão: dept-mês

```
COUNT(DISTINCT username) WHERE is_potential_user = TRUE
(streak ≥ 3 meses consecutivos com is_data_user = TRUE)
```

data_driven_users
Grão: dept-mês

```
COUNT(DISTINCT username) WHERE is_data_user = TRUE no mês
```

share_per_ddi_classification
Grão: classif-dept-mês

```
100 * potential_users / NULLIF(SUM(potential_users) OVER
  (PARTITION BY date_month, site, division, department), 0)
```

coverage_rate
Grão: dept-mês

```
100 * collaborators_with_ddi / NULLIF(active_collaborators_total, 0)
```

transition_rate
Grão: classif-dept-mês

```
(potential_users - LAG(potential_users)) / NULLIF(LAG(potential_users), 0) * 100
  OVER (PARTITION BY site, division, department, user_classification ORDER BY date_month)
```

gap_to_target
Grão: dept-mês

```
potential_users - PERCENTILE_CONT(0.5) WITHIN GROUP
  (ORDER BY total_potential_in_dept) da mesma (date_month, site, division)
```

SQL — Implementação da Regra de 3 Meses Consecutivos

```sql
WITH base AS (
    SELECT
        username,
        date_month,
        is_data_user,
        ROW_NUMBER() OVER (PARTITION BY username ORDER BY date_month)
      - ROW_NUMBER() OVER (PARTITION BY username, is_data_user ORDER BY date_month) AS grp
    FROM stg_ddi          -- já deduplicado (DISTINCT ON)
),
streaks AS (
    SELECT
        username,
        date_month,
        COUNT(*) OVER (PARTITION BY username, grp) AS streak
    FROM base
    WHERE is_data_user IS TRUE   -- só linhas verdadeiras formam a sequência
)
SELECT
    username,
    date_month,
    CASE WHEN streak >= 3 THEN TRUE ELSE FALSE END AS is_potential_user
FROM streaks;

-- Arquivo: transformation/dbt_projects/models/intermediate/int_user_streak.sql
```

1 **Duplo ROW_NUMBER**

rn_all − rn_mesmo_flag = grp. Cada série ininterrupta de TRUE recebe um grp único por usuário. Séries separadas por FALSE recebem grps diferentes.

2 **COUNT(*) OVER (PARTITION BY username, grp)**

Calcula o tamanho de cada sequência consecutiva. Uma série de 5 meses TRUE produz streak = 5 em todas as 5 linhas.

3 **streak ≥ 3 → is_potential_user = TRUE**

A partir do 3º mês da mesma sequência o usuário é marcado como potencial. O próprio mês corrente já conta — não é necessário esperar o 4º mês.

⚠ **Quebra de sequência requer reinício**

Um único mês FALSE zera o grp. O usuário precisa acumular 3 meses TRUE novamente para recuperar o status de potencial.

Diagnóstico – Decisões que o dashboard atual não suporta

① Coverage invisível

Sem coverage_rate por unidade, não é possível distinguir "colaborador sem DDI por falha de integração" de "colaborador genuinamente sem uso de dados".

② Elegibilidade não auditável

A exclusão de Representantes/Shipping não está versionada nem testada. Mudanças silenciosas quebram a comparabilidade MoM do denominador.

③ Funil de maturidade ausente

Mostrar o flag is_data_user do mês corrente não revela quem está a 1 mês de virar Potencial nem quem acabou de entrar na sequência de 3 meses.

④ Migração entre classificações

Times de L&D precisam do transition_rate (Beginner→Intermediate etc.) para priorizar intervenções. O dashboard atual só mostra a fatia estática.

⑤ Ausência de benchmark

Sem gap_to_target, um líder não sabe se "80% data users" é bom ou ruim para aquele site/divisão. É preciso uma referência interna (mediana da divisão).

⑥ Drill hierárquico inconsistente

Empresa→Site→Divisão→Dept com definições diferentes de headcount vs. DDI snapshot distorcem a análise ao detalhar causas de variação.

Decisão: cada lacuna foi resolvida como métrica explícita ou regra versionada no novo produto (agg_ddi_month + dbt tests).

Modelagem

Modelagem • Design Alvo para BI (Validação) • Qualidade e Performance

Camadas Medallion & Granularidade Silver/Gold

RAW	STAGING	INTERMEDIATE	MARTS
tb_people_history tb_ddi (CSV)	stg_people_history (view)	int_people_monthly (elegibil. mensal)	fct_user_month Silver: 1 row/user-mês
Ingestão Airflow	stg_ddi (view) + DISTINCT ON dedup	int_user_streak (streak 3 meses)	agg_ddi_month Gold: agregado
schema: public	not_null tests	Exclusão Ship/Rep	7 métricas finais

Silver — fct_user_month (1 linha por username + date_month)

Colunas: username, date_month, site, division, department (congelados no mês), user_classification, is_data_user, is_potential_user, has_ddi_row, is_active.

SCD: para cada (username, date_month) pega o tim_day MÁXIMO em tb_people_history ≤ último dia de M que satisfaz ativo + não excluído → resolve trocas de divisão mid-month de forma determinística.

Gold — agg_ddi_month (agregado por date_month + site + division + dept + classif)

Colunas de dimensão: date_month, site, division, department, user_classification.

Métricas calculadas: active_collaborators, potential_users, data_driven_users, share_per_ddi_classification, coverage_rate, transition_rate (LAG MoM), gap_to_target (vs. mediana divisão via PERCENTILE_CONT).

Validação Mensal - Duplicidade + Classificação)

1

Unicidade username + date_month

Testes dbt (schema.yml) + pytest (tests/assert_fct_user_month_unique.sql) verificam que nenhum par (username, date_month) aparece mais de uma vez em fct_user_month e em stg_ddi pós-dedup.

2

not_null em chaves e dimensões

Campos username, date_month, site, division, department, user_classification marcados como not_null no schema.yml. Pipeline falha e alerta se qualquer nulo for encontrado no mart.

3

accepted_values para user_classification

Domínio válido: {Champion, Advanced, Intermediate, Basic, Beginner, Sleeper, No Use, Sem registro DDI}. Qualquer valor fora desse conjunto levanta um erro de teste dbt antes de chegar ao BI.

4

Sanidade de coverage_rate

Teste de range: coverage_rate BETWEEN 0 AND 100 por dept-mês. Valores acima de 100% indicam problema no denominador (ativos < registros DDI), o que só ocorre com bug de deduplicação.

5

Reconciliação interna (consistência)

SUM(active_collaborators) agrupado por user_classification dentro do mesmo (date_month, site, division, department) deve igualar COUNT(DISTINCT username) no fct_user_month para o mesmo grupo. Detecta agregações erradas.

Práticas para Eficiência e Baixo Custo

Performance

🗂️ Particionamento por date_month (evolução)

Quando o histórico ultrapassar 12 meses, particionar fct_user_month e agg_ddi_month por date_month elimina full-table scans nas queries de BI, que sempre filtram por mês.

🔍 Índices estratégicos no PostgreSQL

Índice composto em (date_month, site) para filtros do dashboard; índice em (username, date_month) para o streaks join. Ambos cobrem os padrões de acesso dominantes sem overhead de escrita excessivo.

📐 Colunas mínimas nos marts

Marts expõem apenas as colunas consumidas pelo BI. Zero SELECT * nos modelos intermediários. Reduz I/O de disco e latência de serialização nas queries analíticas.

⚡ dbt incremental no Gold (evolução)

Quando a tabela agg_ddi_month crescer além de ~2 anos, migrar para materialização incremental: processar apenas o mês corrente e fazer MERGE/INSERT poupando reprocessamento completo.

💾 Cache no Streamlit com TTL

@st.cache_data com TTL de ~10 min para queries pesadas (evolução 12 meses). Evita reprocessamento a cada interação do usuário mantendo os dados atualizados dentro da janela de atualização mensal.

🔄 Views para staging (sem materialização)

Camadas stg_* e int_* são views, não tabelas. O custo de storage fica nos marts. Para volumes maiores, converter para tables + refresh periódico via dbt run --select stg_*.

SQL

Tabela Agregada • Métricas • Conceitual

Tabela mensal agregada com métricas (**agg_ddi_month**)

```sql
SELECT
    -- ① Dimensões (3.1.1)
    date_month,
    site,
    division,
    department,
    user_classification,

    -- ② Métricas (3.1.2)
    active_collaborators,
    potential_users,
    data_driven_users,
    share_per_ddi_classification,   -- % potenciais por classif no dept-mês
    coverage_rate,                  -- % ativos com registro DDI no mês
    transition_rate,                -- Δ% potential_users MoM (LAG)
    gap_to_target                   -- vs. mediana dept na divisão

FROM analytics.agg_ddi_month

ORDER BY date_month, site, division, department, user_classification;
```

active_collaborators

COUNT(DISTINCT username) no grão pós-elegibilidade

potential_users

COUNT(DISTINCT CASE WHEN is_potential_user THEN username END)

data_driven_users

COUNT(DISTINCT CASE WHEN is_data_user THEN username END)

share_per_ddi_classification

potential_users * 100.0 / NULLIF(SUM(potential_users) OVER (PARTITION BY date_month, site, division, department), 0)

coverage_rate

collaborators_with_ddi * 100.0 / NULLIF(active_collaborators_total, 0)

transition_rate

(potential_users - LAG(potential_users)) / NULLIF(LAG(potential_users), 0) * 100 OVER (PARTITION BY site, division, department, user_classification ORDER BY date_month)

gap_to_target

potential_users - PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_potential_in_dept) particionado por (date_month, site, division)

COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)

Exemplo hipotético: duplicidade em tb_ddi para um mesmo mês

username	date_month	user_classification	is_data_user
← linha duplicada (mesma chave, classificação diferente)			
u1	2025-03-01	No Use	FALSE
u2	2025-03-01	Basic	TRUE
u3	2025-03-01	(NULL)	TRUE

Resultados para a query: SELECT… FROM tb_ddi WHERE date_month = '2025-03-01'

`COUNT(*)`	= 4	Conta todas as linhas, incluindo a duplicata e a linha com NULL.
`COUNT(user_classification)`	= 3	Ignora o NULL na linha de u3. Conta apenas linhas onde user_classification IS NOT NULL.
`COUNT(DISTINCT username)`	= 3	u1 aparece 2×, mas é contado 1× (u1, u2, u3). Nulos em username seriam também ignorados.
`COUNT(DISTINCT user_classification)`	= 3	Valores distintos não nulos: Beginner, No Use, Basic. NULL de u3 não entra na contagem.

Efeito, Detecção e Correção

Duplicidade username + date_month

Efeito nas métricas

COUNT(*) — Inflado

Cada linha duplicada conta como um usuário extra. active_collaborators e data_driven_users ficam maiores que o real.

COUNT(DISTINCT username) — Correto

Corrige a contagem de usuários únicos. Mas não resolve o problema se a duplicata tiver classificações diferentes.

share_per_ddi_classification — Distorcido

Se u1 tem 2 linhas (Beginner + No Use), ambas entram no cálculo de share. A distribuição percentual por classificação fica errada.

Detecção

```sql
SELECT username, date_month, COUNT(*) AS n

FROM tb_ddi

GROUP BY 1, 2

HAVING COUNT(*) > 1;

-- Retorna todos os pares com > 1 linha
```

Correção — 2 abordagens

```sql
-- ① Curadoria na staging (solução adotada)
SELECT DISTINCT ON (username, date_month)
    username,
    date_month,
    user_classification,
    is_data_user
FROM tb_ddi
ORDER BY username, date_month, is_data_user DESC;  -- TRUE > FALSE
-- ② Preferencial: corrigir o pipeline
-- que grava tb_ddi na origem.
```

⚠️ Por que não basta COUNT(DISTINCT username)?

COUNT(DISTINCT username) corrige a contagem de pessoas mas não elimina as linhas extras. Se as duplicatas tiverem user_classification diferentes, o share_per_ddi_classification continua distorcido — a mesma pessoa entra em dois buckets diferentes.

WHERE vs. ON em LEFT JOIN

❌ **WHERE no lado direito — comportamento INNER JOIN**

```sql
SELECT p.username, d.user_classification
FROM stg_people_history p
LEFT JOIN tb_ddi d
    ON p.username = d.username
   AND d.date_month = '2025-06-01'
WHERE d.user_classification = 'Beginner';
-- Elimina linhas onde d.user_classification IS NULL
-- (ou seja, usuários sem DDI no mês somem do resultado)
-- O LEFT JOIN vira efetivamente um INNER JOIN
```

✅ **Filtro no ON — preserva todos da tabela esquerda**

```sql
SELECT p.username, d.user_classification
FROM stg_people_history p
LEFT JOIN tb_ddi d
    ON p.username = d.username
   AND d.date_month = '2025-06-01'
   AND d.user_classification = 'Beginner';
-- Usuários sem DDI Beginner no mês: user_classification = NULL
-- Usuários com DDI Beginner: user_classification = 'Beginner'
-- TODOS aparecem — o LEFT JOIN é preservado
```

Regra de ouro e casos de uso

WHERE no lado direito

Use APENAS quando você quer excluir as linhas que não fizeram match (= comportamento INNER).
Ex: quero apenas colaboradores que TÊM DDI naquele mês.

Filtro no ON

Use quando quer restringir os dados da tabela direita MAS ainda assim preservar todos os registros da tabela esquerda.
Ex: quero todos os colaboradores, mostrando NULL para os sem DDI Beginner.

Caso DDI: coverage_rate

Para calcular coverage_rate precisamos de TODOS os ativos no denominador, inclusive os sem DDI. O join com tb_ddi deve colocar o filtro de mês no ON, não no WHERE.

Dica prática

Se não tem certeza: sempre coloque filtros de atributo da tabela direita no ON. O WHERE final deve filtrar apenas colunas da tabela esquerda ou condições que você quer realmente excluir.

Registro mais recente ‘**Beginner**’ por usuário (1 por usuário)

```sql
SELECT DISTINCT ON (username)
    username,
    date_month,
    user_classification,
    is_data_user
FROM tb_ddi
WHERE user_classification = 'Beginner'
ORDER BY username, date_month DESC;

-- DISTINCT ON (username): mantém somente 1 linha por username
-- ORDER BY username, date_month DESC: a 1ª linha que aparece
--   para cada username é a de date_month mais recente
-- WHERE filtra antes: só considera meses com classif 'Beginner'
```

Exemplo com dados sintéticos

```sql
-- Entrada (todas as linhas Beginner de user_0042):
-- user_0042 | 2025-02-01 | Beginner | TRUE
-- user_0042 | 2025-05-01 | Beginner | TRUE
-- user_0042 | 2025-09-01 | Beginner | FALSE

-- Resultado com DISTINCT ON:
-- user_0042 | 2025-09-01 | Beginner | FALSE  ← mais recente
```

Como DISTINCT ON funciona

Sintaxe exclusiva do PostgreSQL

DISTINCT ON não existe no SQL padrão. Em outros bancos usa-se ROW_NUMBER() OVER (PARTITION BY username ORDER BY date_month DESC) = 1.

ORDER BY deve começar com a coluna do DISTINCT ON

O ORDER BY precisa ter username primeiro (mesmo grupo) e date_month DESC em seguida (critério de seleção). Sem isso, o PostgreSQL retorna erro.

WHERE antes do DISTINCT

O WHERE user_classification = 'Beginner' filtra ANTES da deduplicação. Só as linhas Beginner são consideradas para encontrar a mais recente.

Alternativa com ROW_NUMBER (portável)

SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY username ORDER BY date_month DESC) rn FROM tb_ddi WHERE user_classification = 'Beginner') t WHERE rn = 1

Visualização & Storytelling

Mockup/ Wireframe • Onde estamos? – O que mudou? – Por que mudou? • Microsoft Power BI

Estrutura Narrativa e Decisões de UX

Onde estamos?

Página 1 — Overview

- Filtros no topo: Site ▾ · Divisão ▾ · Mês ▾ · Departamento ▾
- 4 KPI cards: Colaboradores Ativos · Potenciais · Data-Driven Users · Coverage Rate
- Barra horizontal por classificação DDI (Basic, Beginner … Champion, No Use, Sleeper)
- Mini-tabela de DDI% por Site com codificação verde/laranja/vermelho
- Drill: clicar em site filtra toda a página para aquele site

O que mudou?

Página 2 — Evolução

- 4 delta cards: Δ MoM de Potenciais · D-Users · Coverage Rate · Δ acumulado 3M
- Gráfico de linha: Potenciais + Data-Driven Users Jan–Dez com marcadores mensais
- Tabela lateral: Mês | Potenciais | D-Users | Δ MoM (colorido)
- Toggle de horizonte: MoM · 3M · 6M · 12M para o gráfico de linha
- Contexto automático: destaque do mês de pico e do mês atual

Por que mudou?

Página 3 — Diagnóstico

- Gráfico de barras horizontais: Potenciais por Divisão (ordenado decrescente)
- Tabela gap_to_target: Divisão · Δ vs mediana · Classificação que mais impulsionou
- Heatmap site × classificação: intensidade = share_per_ddi_classification
- Decomposição: clicar em divisão expande departamentos com transition_rate MoM
- Annotation automática: divisões que mais contribuíram para a variação do mês

Princípios: hierarquia visual (KPI → tendência → causa) · máximo 4 interações para qualquer insight · codificação de cor consistente entre páginas · benchmark sempre visível

Boas práticas de UX & Data Storytelling Aplicadas

🎯 Contexto antes de detalhe

O usuário vê primeiro o KPI geral da empresa. Só então drilla para site → divisão → departamento. Cada nível adiciona contexto, nunca substitui o anterior.

📊 Benchmark sempre visível

gap_to_target mostra a diferença vs. mediana da divisão em todos os níveis. O usuário nunca precisa perguntar 'isso é bom ou ruim?' — a resposta está no slide.

🔄 Variação MoM explícita

Δ MoM e Δ 3M são exibidos como cards de destaque, não calculados pelo usuário. Cor verde/vermelho comunica direção instantaneamente sem leitura de tabela.

🏷️ Classificações em inglês (como na fonte)

Manter Champion, Advanced, Intermediate, Basic, Beginner, Sleeper, No Use evita ambiguidade de tradução e facilita cruzamento com outros relatórios do Meli.

⚡ Poucas interações para qualquer resposta

Qualquer pergunta de negócio deve ser respondida em ≤ 4 cliques: ① Escolher mês → ② Escolher site → ③ Ver divisão → ④ Ver departamento. Sem menus escondidos.

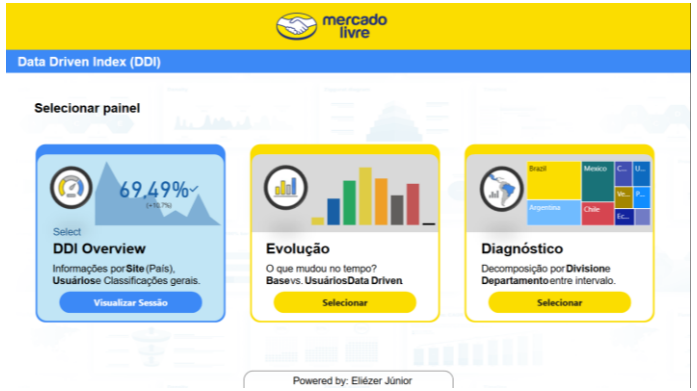
📌 Annotation automática nos gráficos

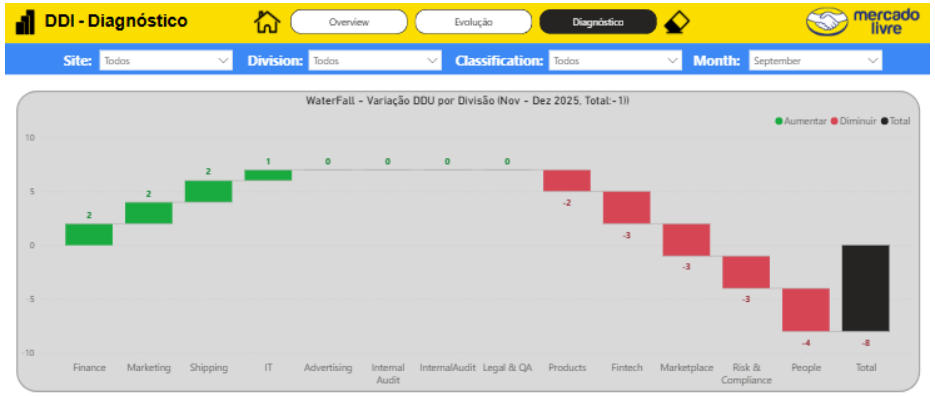
O maior outlier (divisão com maior Δ MoM positivo/negativo) é automaticamente anotado no gráfico de linha. Elimina o trabalho de interpretação básica do analista.

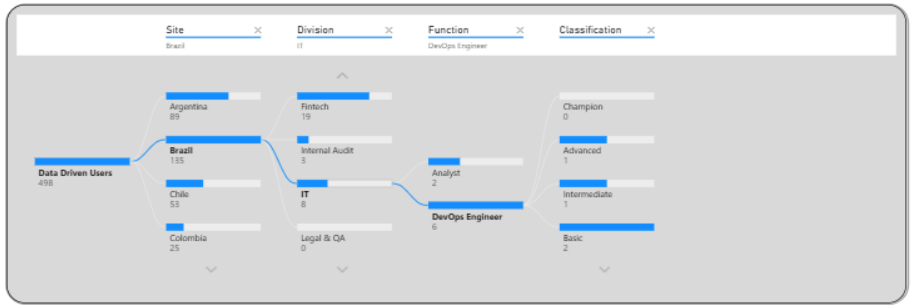
Implementação de referência: serving/streamlit/pages/ · Power BI: serving/powerbi/Data_Driven_Index_DDI.pbix

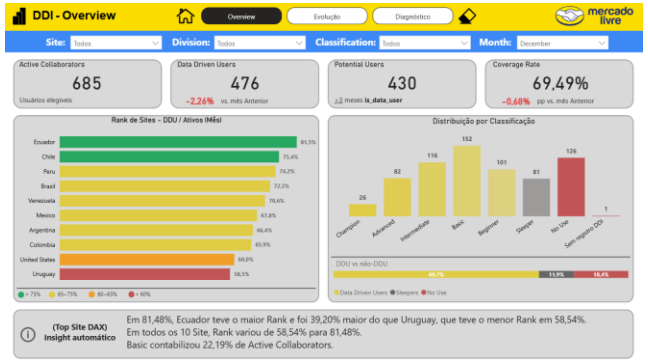
Microsoft Power Bi

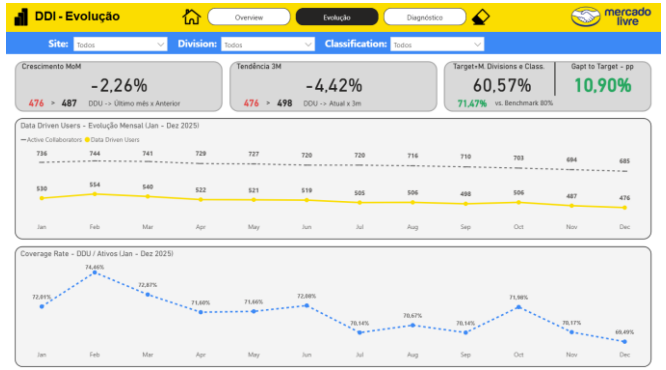
https://app.powerbi.com/view?r=eyJrIjoiZmU4Mjg0M2QtNmM5ZS00MjgyLTg4MzMtYjU4MTJmMzU2YmJliwidCI6ImM3M2JkNjAwLTUxNzMtNDAxZC05YmRjLTUzZTg1YTI5NGNiMiJ9











HEATMAP (Matrix) - Division x Classification - Active Collaborators (Dez 2025)

Division	DDU	Champion	Advanced	Intermediate	Basic	Beginner	Sleeper	No Use	Active
Marketplace	105	7	19	28	20	31	19	30	155
Shipping	57	3	8	12	22	12	7	18	82
Fintech	65	2	8	15	18	22	9	14	89
IT	47	3	5	12	15	13	11	14	73
Products	46	2	7	18	9	10	6	13	66
Risk & Compliance	30	2	3	7	12	6	2	10	42
Advertising	51	3	10	12	16	10	6	9	67
Marketing	39	1	9	6	12	11	8	8	56
People	24	0	2	8	8	6	6	6	36
Finance	22	0	3	7	8	4	3	2	27
Internal Audit	9	1	1	1	4	2	1	2	12
Legal & QA	3	0	1	1	1	0	1	1	5

Mapeamento de Entregáveis x Repositório

Diagnóstico do dashboard atual	`docs/Respostas.md` · Seção 1.1.1 · Slide 2 da defesa
Universo, elegibilidade e exclusões	`transformation/dbt_projects/models/intermediate/int_people_monthly.sql`
Dicionário de 7 métricas	`docs/metrics_layer.md` · `agg_ddi_month.sql` · Slide 4 da defesa
SQL — 3 meses consecutivos	`models/intermediate/int_user_streak.sql` · Slide 5 da defesa
Camadas Medallion + granularidade Silver/Gold	`transformation/dbt_projects/models/` · `docs/architecture.md`
Qualidade (duplicidade + accepted_values)	`models/**/schema.yml` · `tests/assert_*.sql` · `tests/test_warehouse_integration.py`
Performance e boas práticas	`docs/decisions.md` · `docker/sql/bootstrap_ddl.sql` (índices)
SQL 3.1.1 tabela agregada com dimensões	`models/marts/agg_ddi_month.sql` (SELECT das dimensões)
SQL 3.1.2 métricas calculadas	`models/marts/agg_ddi_month.sql` (CTEs com_cov, with_share, final)
COUNT / Duplicidade / WHERE vs ON / DISTINCT ON	`docs/Respostas.md` · Seção 3.2 · Slides 9–11 da defesa
Mockup / wireframe do dashboard	`serving/streamlit/pages/` · `serving/powerbi/` · Slides 10–12 da defesa
Orquestração mensal	`orchestration/airflow/dags/ddi_pipeline.py` · `docker/docker-compose.yml`

Respostas Completas

DATA DRIVEN INDEX

4 seções respondidas	12 questões cobertas	7 métricas especificadas
6 SQLs implementados	3 páginas de dashboard	dbt + Airflow + Docker

Repositório: https://github.com/ersjunior/case_meli_ddi • Dúvidas à disposição